

LSTM Autoencoder-Based Anomaly Detection

Assignment 2 – DV2586 Deep Learning

Petter Eriksson

Blekinge Institute of Technology

May 24, 2026

Contents

1	Introduction	3
2	Dataset and Preprocessing	3
2.1	Dataset Description	3
2.2	Preprocessing Steps	3
3	Model Design	4
3.1	LSTM Autoencoder Model 1	4
3.2	LSTM Autoencoder Model 2	4
4	Training Configuration	4
5	Anomaly Detection Strategy	5
6	Evaluation Metrics	5
7	Results	6
7.1	Training Loss Comparison	6
7.2	Reconstruction Error Visualization	6
7.3	Final Evaluation Results	7
8	Baseline Model Analysis	7
9	Discussion	7
10	Conclusion	8

11 Reproducibility	8
12 References	8

1 Introduction

Anomaly detection is an important task in machine learning used to identify patterns that deviate from normal behavior. Such anomalies may represent system faults, network intrusions, fraudulent activity, or rare operational events.

In this assignment, two different LSTM autoencoder architectures were implemented and evaluated for anomaly detection on the NASA SMAP/MSL telemetry dataset. Reconstruction error was used to identify anomalous behavior in multivariate sequential sensor data.

Additionally, traditional anomaly detection baseline methods, including Isolation Forest and One-Class SVM, were implemented and compared against the LSTM-based approaches.

2 Dataset and Preprocessing

2.1 Dataset Description

The dataset used in this project was the NASA SMAP/MSL anomaly detection dataset.

The dataset contains multivariate spacecraft telemetry measurements where anomalies correspond to abnormal system behavior. Each telemetry channel consists of sequential sensor measurements together with labeled anomaly intervals.

2.2 Preprocessing Steps

The following preprocessing steps were applied:

- StandardScaler normalization
- Sequence generation for LSTM input
- Sequential window generation with window size 50
- Train-test split

The telemetry data contained 25 features per timestep.

The training data shape was:

(2880, 25)

The test data shape was:

(8640, 25)

After preprocessing and sequence generation, the data was transformed into overlapping sequential windows suitable for LSTM training.

3 Model Design

3.1 LSTM Autoencoder Model 1

Model 1 used a simple LSTM autoencoder architecture consisting of:

- Single-layer LSTM encoder
- Latent bottleneck representation
- Single-layer LSTM decoder
- Fully connected reconstruction layer

The architecture was designed to learn normal sequential behavior and reconstruct normal telemetry sequences.

3.2 LSTM Autoencoder Model 2

Model 2 used a deeper architecture with increased model complexity.

- Two-layer LSTM encoder
- Larger hidden representation
- Dropout regularization
- Two-layer LSTM decoder

The goal of the deeper model was to improve representation learning and anomaly detection performance.

4 Training Configuration

The models were trained using the following configuration:

- Loss function: Mean Squared Error (MSE)
- Optimizer: Adam
- Learning rate: 0.001

- Batch size: 64
- Epochs: 30
- Device: CUDA GPU when available

The models were trained to reconstruct normal telemetry sequences. Lower reconstruction error indicates that a sequence resembles normal behavior, while larger reconstruction errors may indicate anomalies.

5 Anomaly Detection Strategy

Anomalies were detected using reconstruction error.

The reconstruction error was computed as:

$$\text{Reconstruction Error} = \|x - \hat{x}\|^2$$

where x represents the original sequence and $\hat{x}$ represents the reconstructed sequence.

Thresholds were selected using 95th percentile thresholding on the reconstruction error distribution. Sequences above the selected threshold were classified as anomalous.

6 Evaluation Metrics

The following evaluation metrics were used:

- Precision
- Recall
- F1-score
- ROC-AUC

Binary anomaly labels were generated from the provided anomaly intervals in the dataset.

7 Results

7.1 Training Loss Comparison

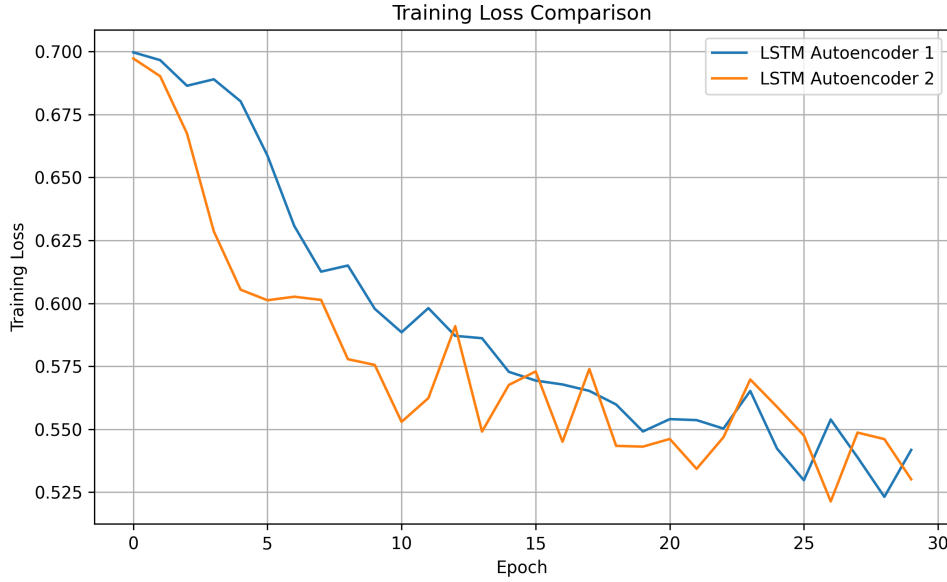


Figure 1: Training loss comparison between LSTM Autoencoder models

The training curves show that both models successfully learned to reconstruct the telemetry sequences. Model 1 converged faster and achieved lower reconstruction loss compared to Model 2.

7.2 Reconstruction Error Visualization

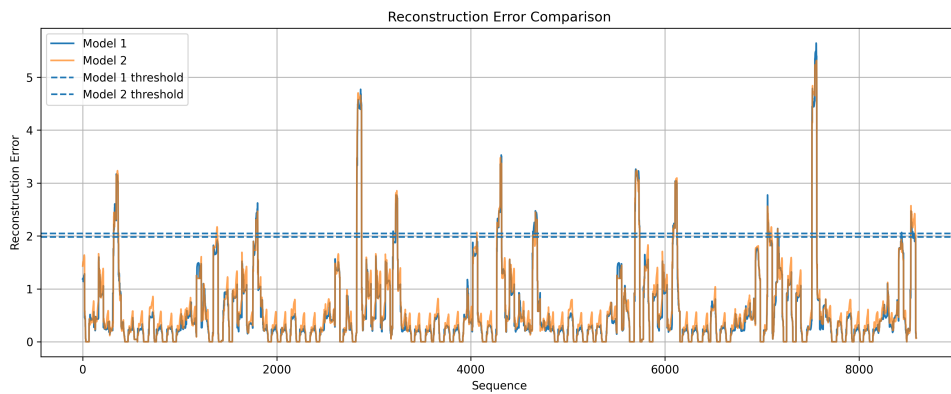


Figure 2: Reconstruction error comparison between the models

Both models produced clear reconstruction error spikes during anomalous regions. Model 1 generated larger anomaly peaks, while Model 2 produced smoother reconstruction behavior.

7.3 Final Evaluation Results

Model	Precision	Recall	F1-score	ROC-AUC
LSTM Autoencoder 1	0.1023	0.5238	0.1712	0.8573
LSTM Autoencoder 2	0.0465	0.2381	0.0778	0.8135
Isolation Forest	0.0000	0.0000	0.0000	0.3253
One-Class SVM	0.0416	0.5357	0.0771	0.7944

Table 1: Final anomaly detection results

LSTM Autoencoder 1 achieved the best overall performance with the highest precision, F1-score, and ROC-AUC.

One-Class SVM achieved the highest recall, meaning it detected the largest portion of the true anomalies, but it also produced more false positives.

8 Baseline Model Analysis

Isolation Forest was initially tested as a baseline anomaly detection model because it is a commonly used unsupervised anomaly detection method and was suggested as one of the possible baseline approaches in the assignment description. However, the model achieved very poor results, with zero precision, recall, and F1-score, as the predicted anomaly locations did not overlap with the labeled anomaly intervals.

One likely reason is that Isolation Forest does not explicitly model temporal dependencies in the telemetry signals. Although the sequential windows were flattened into vectors, much of the temporal structure was lost, making it difficult for the model to identify contextual anomalies in the spacecraft telemetry data.

Due to the poor performance of Isolation Forest, One-Class SVM was selected as the final baseline model. One-Class SVM achieved better results and obtained a higher recall than LSTM Autoencoder 1, meaning it detected a larger portion of the true anomalies. However, its precision and F1-score were still lower than those of LSTM Autoencoder 1.

In contrast, the LSTM autoencoders are specifically designed for sequential time-series data and were therefore better able to capture anomalous temporal behavior. This is also reflected in the higher ROC-AUC and F1-score achieved by the LSTM-based models.

9 Discussion

The results demonstrate that LSTM autoencoders are effective for anomaly detection in sequential spacecraft telemetry data.

The LSTM-based models achieved significantly higher ROC-AUC scores compared to the traditional baseline approaches. This suggests that modeling temporal dependencies is important for detecting anomalies in time-series telemetry data.

Model 1 performed better overall than Model 2 despite being simpler. One possible reason is that the deeper architecture of Model 2 introduced additional optimization difficulty and mild overfitting on the relatively small dataset.

Threshold selection also had a significant effect on the anomaly detection results. Initial thresholding using mean plus standard deviation produced poor anomaly detection performance, while percentile-based thresholding improved the results considerably.

10 Conclusion

This assignment demonstrated the effectiveness of LSTM autoencoders for anomaly detection in multivariate sequential telemetry data.

The best-performing model achieved a ROC-AUC score of approximately 0.87, showing strong ability to separate normal and anomalous behavior.

The comparison with traditional baseline methods highlighted the importance of temporal modeling for anomaly detection tasks involving sequential data.

11 Reproducibility

To reproduce the results:

1. Install the required Python libraries
2. Download the NASA SMAP/MSL dataset
3. Fixed random seeds were used for NumPy, PyTorch, and CUDA to improve reproducibility
4. Run preprocessing and sequence generation
5. Train the models
6. Execute evaluation scripts

12 References

- NASA SMAP/MSL Dataset: <https://www.kaggle.com/datasets/patrickfleith/nasa-anomaly-detection-dataset-smap-msl>
- F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation Forest,” 2008.

- PyTorch Documentation: <https://pytorch.org/docs/stable/index.html>
- Scikit-learn Documentation: <https://scikit-learn.org/stable/>
- M. Ekman, *Learning Deep Learning: Theory and Practice of Neural Networks, Computer Vision, Natural Language Processing, and Transformers Using Tensor-Flow*, Pearson, 2022.